Line-Following Maze Robot
Design and Development by Raunak Choudhary

Full Parameter & Variable Reference

Arduino / TB6612 Motor Driver — Right-Hand-Rule Maze Solver

1. Junction-Detection Flags

These five booleans/int are the core sensing outputs recalculated at every junction. checknode() sets them from live sensor readings, and reposition() reads them to decide which direction to take.

```
bool l = 0;   // Left path exists at current node?
bool r = 0;   // Right path exists at current node?
bool s = 0;   // Straight path exists at current node?
bool u = 0;   // Dead end / turn-around needed?
bool x = 0;   // Reset every node, never used elsewhere
bool y = 0;   // Reset every node, never used elsewhere
int  e = 0;   // End-of-maze flag; becomes 2 when end marker is detected
```

Name	Type	Purpose
l	bool	True when the left-side sensor reading indicates an open path to the left at the current node.
r	bool	True when the right-side sensor reading indicates an open path to the right at the current node.
s	bool	True when the center sensor indicates the line continues straight ahead.
u	bool	True when no other exit is found — signals a dead end requiring a U-turn.
x	bool	Declared and reset each node in checknode(), but never read or acted on anywhere else — dead variable.
y	bool	Same as x — declared, reset, never used. Dead variable.
e	int	Set to 2 when all relevant sensors read 'black' simultaneously, which

Name	Type	Purpose
		is interpreted as the maze end marker.

Why these matter

l, r, s, and u together describe the shape of the current junction (how many exits it has and where). This is the raw sensing layer that everything else — path logging, the right-hand rule, and turning decisions — is built on top of.

2. "Custom" Variables (Currently Unused)

These two variables are declared under a // custom variable comment but are never referenced anywhere else in the file. They appear to be leftovers from an earlier version of the logic — perhaps intended to track which side the robot last exited from.

```
bool leftOut = 0;   // Declared, never referenced again
bool rightOut = 0;  // Declared, never referenced again
```

Name	Type	Purpose
`leftOut`	bool	No functional effect on the program — safe to remove if simplifying the code.
`rightOut`	bool	Same — dead code with no functional effect.

3. Path-Tracking Variables

```
int  paths = 0;      // Count of open exits at a node
bool endFound = 0;   // True once the maze end has been located
```

Name	Type	Purpose
`paths`	int	Sum of open exits at the current node, computed as l + s + r inside checknode(). Used to decide whether a junction is significant enough to log to str (single-path corridors are not logged as decisions).
`endFound`	bool	Flips to true inside reposition() once e == 2 is detected. This is the condition that breaks the main maze-exploration while loop in loop().

4. Threshold Constants (Unused)

Both constants below look like they were meant to define fixed reference points for "what counts as black" and "what counts as white," but neither is referenced anywhere in the code. The actual black/white decision is instead made dynamically by the calibrated threshold[] array produced in calib().

```
int blackValue = 900;  // Never referenced anywhere in the code
int whiteValue = 100;  // Never referenced anywhere in the code
```

Name	Type	Purpose
`blackValue`	int	Dead constant. Not used by any function — threshold[i] (computed at runtime during calibration) is used instead.
`whiteValue`	int	Dead constant, same as above.

5. Feeler Time (FT)

```
int FT = 150;   // Number of PID() iterations run inside checknode()
                // while scanning for a branch at a junction
```

FT ("Feeler Time") sets how many PID-correction loop iterations run while the robot is creeping into a junction and checking its side sensors for openings. Since each PID() call takes a small, roughly fixed amount of time, this value effectively controls how far/long the robot advances into the junction before it commits to a decision.

Value	Effect	Trade-off
`Larger FT`	Robot creeps further into the junction before deciding	More reliable branch detection, but slower and risks overshooting on tight maze cells
`Smaller FT`	Robot decides sooner	Faster response, but risks missing a branch if the side sensors trigger late

6. PID Working Variables

```
int P, D, I, previousError, PIDvalue, error;
```

Name	Type	Purpose
P	int	Proportional term for the current cycle — simply equal to error.
I	int	Integral term — running accumulation of error over time (I += error each cycle).
D	int	Derivative term — rate of change of error, computed as error - previousError.
`previousError`	int	Stores the error value from the last PID cycle, needed to compute D on the next cycle.

Name	Type	Purpose
PIDvalue	int	Final blended steering correction: Kp*P + Ki*I + Kd*D, added to one motor's speed and subtracted from the other's.
error	int	Raw sensor imbalance for the current cycle: analogRead(1) - analogRead(3), i.e. left-inner sensor minus right-inner sensor.

Precision caveat

PIDvalue = (Kp * P) + (Ki * I) + (Kd * D) mixes float gains (Kp, Kd) with an int accumulator (PIDvalue). The floating-point result is truncated when stored into the int. Because Kp and Kd are small (0.04 and 0.05), a modest error like 10 produces Kp*P = 0.4, which truncates to 0 — small corrections can silently disappear. Consider making PIDvalue a float, or scaling the gains up and dividing later, to avoid this.

7. Speed Variables

```
int lsp = 150;       // Left motor speed for the current PID cycle
int rsp = 150;        // Right motor speed for the current PID cycle
int lfspeed = 220;     // Base "line-follow" forward speed
int turnspeed;          // Declared here with no value; set to 100 in setup()
```

Name	Initial value	Purpose
lsp	150	Left motor speed computed each PID() call as lfspeed - PIDvalue, clamped to 0–255. The initial 150 is irrelevant since it is overwritten before ever being used.
rsp	150	Right motor speed computed each PID() call as lfspeed + PIDvalue, clamped to 0–255. Same note about the initial value applies.
lfspeed	220	Nominal forward speed both motors aim for during straight-line driving, before PID correction is applied. Also used as the base kick speed for the U-turn maneuver.
turnspeed	(unset here; 100 via setup())	Pivot speed used specifically during botleft(), botright(), and the sustained phase of botuturn().

8. PID Gains

```
float Kp = 0.04;   // Proportional gain
float Kd = 0.05;    // Derivative gain
float Ki = 0;        // Integral gain (disabled)
```

Name	Value	Role
Kp	0.04	Proportional gain — scales how strongly the robot reacts to the current error. Higher Kp means sharper, more aggressive correction.
Kd	0.05	Derivative gain — scales how strongly the robot reacts to the rate of change of the error. This damps oscillation and smooths the response.
Ki	0	Integral gain — currently disabled. The accumulated error (I) is still calculated every cycle but has zero effect on steering because it is multiplied by 0.

Tuning notes

- With Ki = 0, this is effectively a PD controller, not a full PID. That is a reasonable choice for line following, since an integral term can cause windup and oscillation on sharp turns.
- Kd (0.05) is slightly larger than Kp (0.04), meaning the controller leans a bit more on damping oscillation than on aggressive correction — favors stability, though it may respond a little sluggishly to sudden line loss.
- Because of the int/float truncation issue noted in Section 6, the effective gain at small errors is lower than these numbers suggest.

9. Full Summary Table

Name	Type / Value	Status & Purpose
l, r, s, u	bool	Live junction-exit flags, set in checknode(), consumed in reposition().
x, y	bool	Reset every node; never used elsewhere. Dead code.
e	int (0 or 2)	Maze-end detector flag.
leftOut, rightOut	bool	Declared, never used anywhere. Dead code.
paths	int	Count of open exits at a node (l + s + r).
endFound	bool	Set true when maze end is found; breaks exploration loop.
blackValue, whiteValue	int (900, 100)	Declared, never used. Real thresholds come from calibration instead.
FT	int (150)	PID-loop iteration count used while feeling out a junction's branches.
P, I, D	int	Standard PID terms; I is calculated but has no effect since Ki = 0.
previousError, error	int	Current and prior sensor imbalance, used to compute D.
PIDvalue	int	Final steering correction, truncated from a float calculation.
lsp, rsp	int (150, 150)	Per-cycle motor speeds; initial values overwritten immediately.
lfspeed	int (220)	Base forward speed.

Name	Type / Value	Status & Purpose
`turnspeed`	int (set to 100 in setup())	Pivot speed for turning maneuvers.
`Kp, Kd, Ki`	float (0.04, 0.05, 0)	PID gains — effectively PD control since Ki = 0.

10. Cleanup Recommendations

- Remove dead variables: x, y, leftOut, rightOut, blackValue, whiteValue. They consume memory and add confusion with no functional benefit.
- Consider making PIDvalue (and possibly P, D, I) a float to avoid truncation of small corrections before the final clamp to an int motor speed.
- Double-check the shared pin 9 between startButton and STBY (see the full code review) — unrelated to this variable list, but worth resolving alongside any cleanup.
- If Ki will remain 0 permanently, consider removing the integral term entirely (I, Ki, and the I += error line) to simplify PID() and save a few CPU cycles per loop.